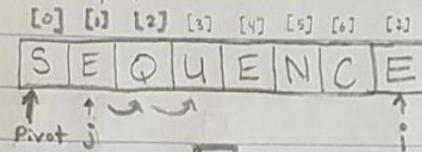


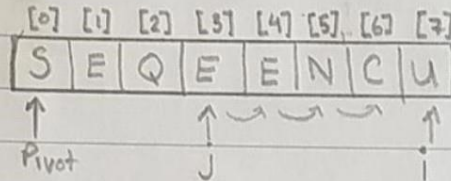
Homework #8 ★ 8.2, 8.4, 8.5, 8.9(d), 8.10

8.2 ★

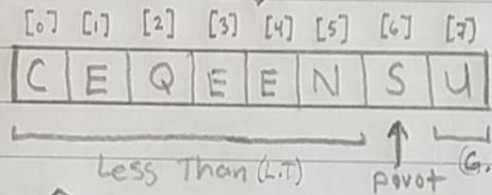
Sort the characters of the string "Sequence" using in-place quicksort. Suppose that every pivot choice chooses the first element in the range.



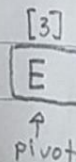
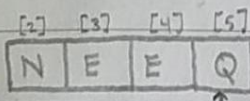
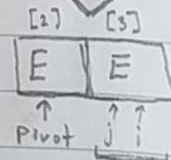
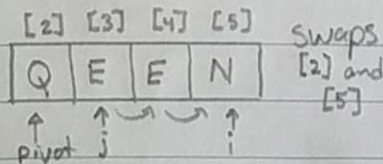
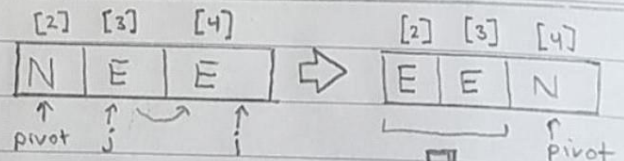
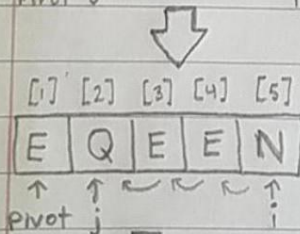
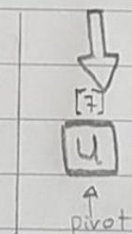
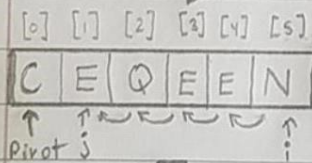
Since all values up to index [3] are less than pivot, we increment j until...



at index 3, U is Greater Than pivot, (S) so we swap j and i. we continue until we reach i and counter j stops.



The pivot is swapped with index i-1, and the array is now partitioned. Now, this process is repeated for the partitions until there is one/no elements left in each sub-partition.

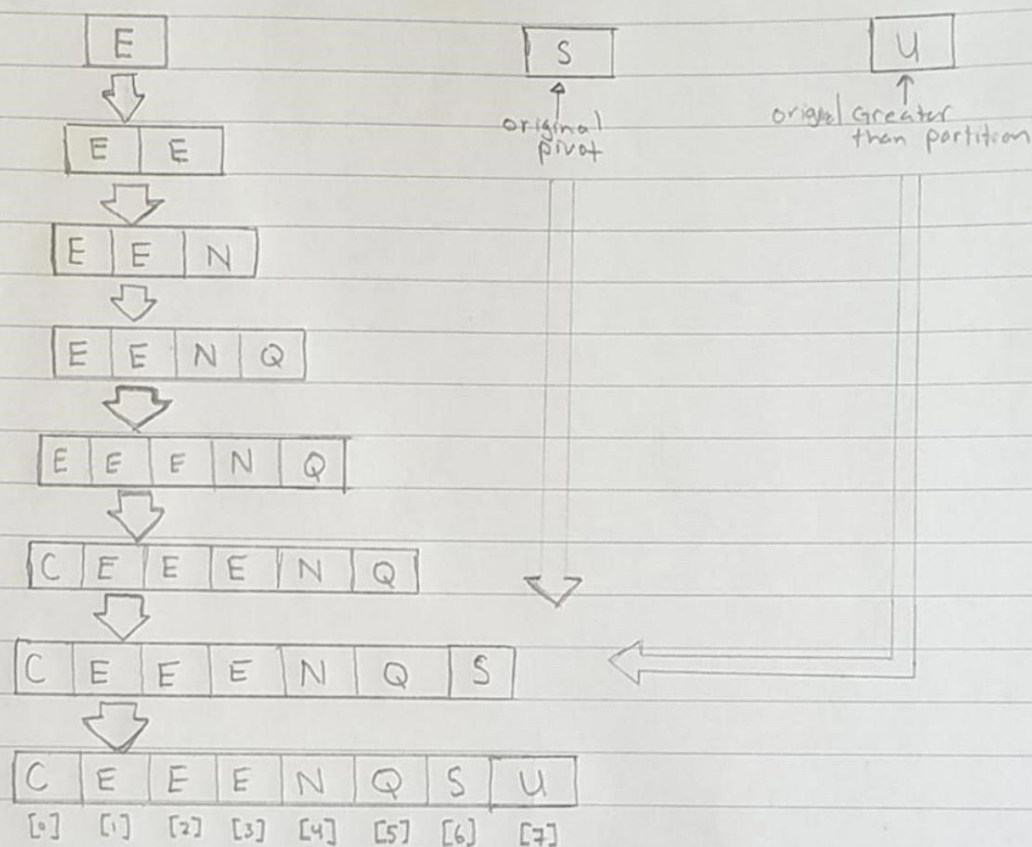


(Since this is in-place,
no sub-arrays are created)

Now that we have "divided," we can CONQUER!

8.2

(Continued)



Algorithm is complete.

8.4

The "PHP hash table collision exploit" makes use of the fact that the hash function used to determine which index or "bucket" a key/value is assigned to could be predicted. Since hash functions were, at the time, mostly focused on speed and uniform distribution of keys to prevent collisions, the predictability of collisions was overlooked.

The exploit takes advantage of this predictability by forcing collisions in a way that makes the usually fast hashing algorithm more complex, thus utilizing more of the machine's resources, namely CPU.

This became known as a "hash-flooding" attack.

The exploit could have been prevented, and eventually was solved, with the use of One-Way Cryptographic Hash functions. Since one-way hash functions are exceedingly difficult to invert, the predictability factor for collisions was solved, at the expense of time.

The trade-off was worth it for the case of hash tables, and by 2013, many programming languages adopted a hash function that was a one-way cryptographic function, namely SipHash.

Source: <https://medium.freecodecamp.org/hash-table-attack-8e4371fc5261>

8.5

$$h(x) = (5x) \bmod 8$$

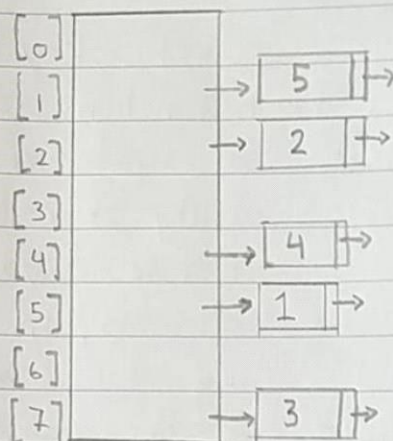
$$1 \rightarrow (1 \cdot 5) \% 8 = 5$$

$$2 \rightarrow (2 \cdot 5) \% 8 = 2$$

$$3 \rightarrow (3 \cdot 5) \% 8 = 7$$

$$4 \rightarrow (4 \cdot 5) \% 8 = 4$$

$$5 \rightarrow (5 \cdot 5) \% 8 = 1$$



Circuit Satisfaction Problem

8.9(d)

input: a Boolean circuit C , with k variables and n vertices

output: a Boolean assignment A that satisfies C , or **NONE** if none

err, I'm not sure how to approach this problem for a general Boolean circuit C , but I'll try.

The only way that I think this can be solved given the problem constraints is to brute-force it by randomly assigning TRUE or FALSE to each variable, testing to see if it satisfies the circuit, and returning the assignment A if the circuit is satisfied. This seems way too complicated for a feasible Monte Carlo solution, so I think I'm not approaching the problem correctly, since my approach would need a helper-function to evaluate each assignment.

8.10

Three-way-Quicksort.

```

void quicksort(int array, int low, int high) {
    assign pivot to highest value in array;
    assign less_than to low;           // less_than is lt henceforth
    assign reader to low;
    assign greater_than to high;      // greater_than is gt henceforth
    while reader < gt {
        if (a[reader] < pivot) {
            swap(a, reader, lt);
            increment reader;
            increment lt;
        }
        else if (a[reader] == pivot) {
            decrement gt;
            swap(a, reader, gt);
        }
        else {
            reader++;
        }
    }
    int minimum = min(gt - lt, high - gt + 1);
    for (int i = 0; i < minimum; increment i) {
        swap(a, lt + i, high - minimum + 1 + i);
    }
    if (low < high) {
        quicksort(a, low, lt - 1); // recursively sort sub-arrays
        quicksort(a, high - gt + 1, high);
    }
    return;
}

```

The time complexity is $\Theta(n^2)$. The expected length of each subarray is $(|A|/3)/2$

I used the following source for help on this problem

<https://geeksforgeeks.org/3-way-quicksort-dutch-national-flag/>